

سوال اول

مربع‌های جادویی آرایه‌هایی مربعی از اعداد بدون تکرار هستند که مجموع هر سطر، ستون و قطر اصلی آن‌ها برابر است. یکی از انواع آن مربع جادویی با کمترین مجموع اعداد است.

در یک مربع جادویی 3×3 که از اعداد متوالی بدون تکرار استفاده می‌شود، کمترین مجموع ممکن برای هر سطر، ستون و قطر عدد ۱۵ است.

الف) این مسئله را در قالب CSP فرموله کنید و در آن متغیرها، دامنه‌ها و محدودیت‌ها را به صورت ضمنی مشخص کنید

ب) چند خانه از مربع جادویی 3×3 زیر مشخص شده است. بقیه خانه‌های این مربع جادویی را به روش جستجو عقب‌گرد (Backtrack search) پر کرده و در هر مرحله، دامنه متغیرها را پس از فیلتر کردن به روش بررسی رو به جلو (Forward Checking) مشخص کرده و بنویسید.

ترتیب پر کردن خانه‌ها، ابتدا بالاترین و سپس چپ‌ترین و ترتیب امتحان کردن مقادیر از ۹ به ۱ می‌باشد.

4		
	5	

پاسخ:

هر خانه مربع را یک متغیر گرفته و به صورت زیر اسم گذاری می‌کنیم.

A_{11}	A_{12}	A_{13}
A_{21}	A_{22}	A_{23}
A_{31}	A_{32}	A_{33}

دامنه:

$$D = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

محدودیت‌ها:

$$A_{11} + A_{12} + A_{13} = 15$$

$$A_{21} + A_{22} + A_{23} = 15$$

$$A_{31} + A_{32} + A_{33} = 15$$

$$A_{11} + A_{21} + A_{31} = 15$$

$$A_{12} + A_{22} + A_{32} = 15$$

$$A_{13} + A_{23} + A_{33} = 15$$

$$A_{11} + A_{22} + A_{33} = 15$$

$$A_{13} + A_{22} + A_{31} = 15$$

$$A_{11} \neq A_{21} \neq A_{31} \neq A_{21} \neq A_{22} \neq A_{23} \neq A_{31} \neq A_{32} \neq A_{33}$$

4			1	2	3	1	2	3
					6			6
			7	8	9	7	8	9
1	2	3	5			1	2	3
		6						6
7	8	9				7	8	9
1	2	3	1	2	3			
		6			6			6
7	8	9	7	8	9			

4			9				2	
1	2	3	5			1	2	3
		6						6
7	8					7	8	
1	2	3	1					
		6						6
7	8							

4			9			2		
1		3	5			1		3
		6						6
7	8					7	8	
			1					
								6
	8							

4			9			2		
1		3	5			1		3
		6						6
7	8					7	8	
			1					
								6
	8							

4			9			2		
8			5					
			1					
								6

Backtrack

4			9			2		
7			5					3
			1					
								6

Backtrack

4			9			2		
6			5					
			1					
								6

Backtrack

4			9			2		
3			5					
						7		
			1					
								6
	8							

4			9			2		
3			5			7		
			1					
								6
	8							

4	9	2
3	5	7
8	1	
		6

4	9	2
3	5	7
8	1	
		6

4	9	2
3	5	7
8	1	6

Q6. [13 pts] Crossword Puzzles as CSPs

You are developing a program to automatically solve crossword puzzles, because you think a good income source for you might be to submit them to the New York Times (\$200 for a weekday puzzle, \$1000 for a Sunday).¹ For those unfamiliar with crossword puzzles, a crossword puzzle is a game in which one is given a grid of squares that must be filled in with intersecting words going from left to right and top to bottom. There are a given set of starting positions for words (in the grid below, the positions 1, 2, 3, 4, and 5), where words must be placed going across (left to right) or down (top to bottom). At any position where words intersect, the letters in the intersecting words must match. Further, no two words in the puzzle can be identical. An example is the grid below, in which the down words (1, 2, and 3) are DEN, ARE, and MAT, while the across words (1, 4, and 5) are DAM, ERA, and NET.

Example Crossword Grid and Solution

¹ D	² A	³ M
⁴ E	R	A
⁵ N	E	T

A part of your plan to make crosswords, you decide you will create a program that uses the CSP solving techniques you have learned in CS 188, since you want to make yourself obsolete at your own job from the get-go. Your first task is to choose the representation of your problem. You start with a dictionary of all the words you could put in the crossword puzzle, where the dictionary is of size K and consists of the words $\{d_1, d_2, \dots, d_K\}$. Assume that you are given a grid with N empty squares and M different entries for words (and there are 26 letters in the English language). In the example above, $N = 9$ and $M = 6$ (three words across and three words down).

You initially decide to use words as the variables in your CSP. Let D_1 denote the first down word, D_2 the second, D_3 the third, etc., and similarly let A_k denote the k th across word. For example, in the crossword above, $A_1 = \text{DAM}$, $D_1 = \text{DEN}$, $D_2 = \text{ARE}$, and so on. Let $D_1[i]$ denote the letter in the i th position of the word D_1 .

(a) [1 pt] What is the size of the state space for this CSP?

Several answers are acceptable for this problem. The simplest is that the dictionary has size K and there are M words, giving state space size K^M . A slightly tighter bound is achieved by noting that once one word is placed, the next words must all be different, giving $K(K-1)(K-2)\cdots(K-M+1) = \frac{K!}{(K-M)!}$. Noticing that we are choosing M distinct words out of a possible K gives the state space bound $\binom{K}{M}$.

Several students tried to include N in their answers; since the letters have nothing to do with this formulation of the problem, this was incorrect. Many students also incorrectly had M^K .

(b) [3 pts] Precisely (i.e. use mathematical notation to) describe the constraints of the CSP when we use words as variables.

For every pair of across and down words D_k and A_l that intersect, we have the constraint that their letters are equal. Specifically, if they intersect in positions i and j , we have $D_k[i] = A_l[j]$.

We also have the pairwise constraints that none of the words are the same: for $k \neq k'$, $D_k \neq D_{k'}$ and $A_k \neq A_{k'}$, and for all k, k' , we have $A_k \neq D_{k'}$.

In addition, each word must have the correct length. One possible formulation is that for all $L \in \mathbb{N}$, for all words D_k and A_l with length L in the puzzle, we have $\text{length}(D_k) = L$ and $\text{length}(A_l) = L$.

The biggest problem that students had was assuming that all crossword puzzles were contiguous squares (or rectangles) like the example. While that works for the above example, it will not work generally. Several students missed one or two of the above constraints, and all three were necessary for full credit. Minor mistakes included missing a few of the inequality constraints.

¹<http://www.nytimes.com/2009/07/19/business/media/19askthetimes.html>

After defining your CSP, you decide to go ahead and make a small crossword using the grid below. Assume that you use the words on the right as your dictionary.

Crossword Grid

1	2	3	4	
5				
6				
7				

Dictionary Words

ARCS, BLAM, BEAR, BLOGS, LARD, LARP,
GAME, GAMUT, GRAMS, GPS, MDS, ORCS, WARBLER

(c) [1 pt] Enforce all *unary* constraints by crossing out values in the table below.

D_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_2	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_3	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_4	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_5	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_6	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_7	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER

Here's an extra table in case you make a mistake:

D_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_2	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_3	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_4	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_5	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_6	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_7	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER

(d) [1 pt] Assume that in backtracking search, we assign A_1 to be GRAMS. Enforce unary constraints, and in addition, cross out all the values eliminated by forward checking against A_1 as a result of this assignment.

D_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_2	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_3	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_4	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_5	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_6	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_7	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER

Here's an extra table in case you make a mistake:

D_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_2	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_3	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_4	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_5	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_6	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_7	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER

- (e) [3 pts] Now let's consider how much arc consistency can prune the domains for this problem, even when no assignments have been made yet. I.e., assume no variables have been assigned yet, enforce unary constraints first, and then enforce arc consistency by crossing out values in the table below.

D_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_2	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_3	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
D_4	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_1	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_5	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_6	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER
A_7	ARCS	BLAM	BEAR	BLOGS	LARD	LARP	GPS	MDS	GAME	GAMUT	GRAMS	ORCS	WARBLER

The common mistake in this question was to leave a few blocks of words that students thought could not be eliminated. Probably the most common was to allow both LARD and LARP for D_2 and A_5 . This is incorrect; for D_2 , no assignment of A_7 is consistent with LARP, and for A_5 , no assignment of D_4 is consistent with LARD.

- (f) [1 pt] How many solutions to the crossword puzzle are there? Fill them (or the single solution if there is only one) in below.

¹ B	² L	³ O	⁴ G	S
⁵ L	A	R	P	
⁶ A	R	C	S	
⁷ M	D	S		

1	2	3	4	
5				
6				
7				

1	2	3	4	
5				
6				
7				

There is one solution (above)

Your friend suggests using letters as variables instead of words, thinking that sabotaging you will be funny. Starting from the top-left corner and going left-to-right then top-to-bottom, let X_1 be the first letter, X_2 be the second, X_3 the third, etc. In the very first example, $X_1 = D$, $X_2 = A$, and so on.

- (g) [1 pt] What is the size of the state space for this formulation of the CSP?

26^N . There are 26 letters and N possible positions.

- (h) [2 pts] Assume that in your implementation of backtracking search, you use the least constraining value heuristic. Assume that X_1 is the first variable you choose to instantiate. For the crossword puzzle used in parts (c)-(f), what letter(s) might your search assign to X_1 ?

We realized that this question was too vague to be answered correctly, so we gave everyone 2 points for the problem. The least constraining value heuristic, once a variable has been chosen, assigns the value that according to some metric (chosen by the implementer of the heuristic) leaves the domains of the remaining variables most open. How one eliminates values from the domains of other variables upon an assignment can impact the choice of the value as well (whether one uses arc consistency or forward checking).

We now sketch a solution to the problem assuming we use forward checking. Let $X_1, X_2, \dots, X_5$ be the letters in the top row of the crossword and X_1, X_6, X_7, X_8 be the first column down. Upon assigning $X_1 = G$, the possible domains for the remaining letters are

$$X_2 \in \{A, R\}, X_3 \in \{M, A\}, X_4 \in \{U, M\}, X_5 \in \{T, S\}, X_6 \in \{A\}, X_7 \in \{M\}, X_8 \in \{E\}.$$

Upon assigning $X_1 = B$, the possible domains remaining are

$$X_2 \in \{L\}, X_3 \in \{O\}, X_4 \in \{G\}, X_5 \in \{S\}, X_6 \in \{L, E\}, X_7 \in \{A\}, X_8 \in \{M, R\}.$$

The remaining variables are unaffected since we are using only forward checking. Now, we see that with the assignment $X_1 = G$, the minimum size remaining for any domain is 1, while the sum of the sizes remaining domains is 11; for $X_1 = B$, the minimum size is 1, while the sum of the sizes remaining is 9. So depending on whether we use minimum domain or the sum of the sizes of the remaining domains, the correct solutions are G and B or only G, respectively.

Any choice but $X_1 = B$ or $X_1 = G$ will eliminate all values for one of the other variables after forward checking.

4. (21 points) Time Management

Two of our GSIs, Arjun and Woody, are making their schedules for a busy morning. There are five tasks to be carried out:

- (F) Pick up food for the group's research seminar, which, sadly, takes one precious hour.
- (H) Prepare homework questions, which takes 2 consecutive hours.
- (P) Prepare the PR2 robot for a group of preschoolers' visit, which takes one hour.
- (S) Lead the research seminar, which takes one hour.
- (T) Teach the preschoolers about the PR2 robot, which takes 2 consecutive hours.

The schedule consists of one-hour slots: 8am-9am, 9am-10am, 10am-11am, 11am-12pm. The requirements for the schedule are as follows:

- (a) In any given time slot each GSI can do at most one task (F, H, P, S, T).
- (b) The PR2 preparation (P) should happen before teaching the preschoolers (T).
- (c) The food should be picked up (F) before the seminar (S).
- (d) The seminar (S) should be finished by 10am.
- (e) Arjun is going to deal with food pick up (F) since he has a car.
- (f) The GSI not leading the seminar (S) should still attend, and hence cannot perform another task (F, T, P, H) during the seminar.
- (g) The seminar (S) leader does not teach the preschoolers (T).
- (h) The GSI who teaches the preschoolers (T) must also prepare the PR2 robot (P).
- (i) Preparing homework questions (H) takes 2 consecutive hours, and hence should start at or before 10am.
- (j) Teaching the preschoolers (T) takes 2 consecutive hours, and hence should start at or before 10am.

To formalize this problem as a CSP, use the variables F, H, P, S and T. The values they take on indicate the GSI responsible for it, and the starting time slot during which the task is carried out (for a task that spans 2 hours, the variable represents the starting time, but keep in mind that the GSI will be occupied for the next hour also - make sure you enforce constraint (a)!). Hence there are eight possible values for each variable, which we will denote by A8, A9, A10, A11, W8, W9, W10, W11, where the letter corresponds to the GSI and the number corresponds to the time slot. For example, assigning the value of A8 to a variable means that this task is carried about by Arjun from 8am to 9am.

- (a) (2 pt) What is the size of the state space for this CSP?

8^5 since every task variable has 8 values, 4 time slots $\times$ 2 GSIs to carry them out, and there are 5 such tasks.

- (b) (2 pt) Which of the statements above include unary constraints?

A unary constraint constrains the domain of a single variable. (d), (e), (i), (j) are unary constraints. (i) and (j) express both unary constraints (on the time of the tasks) and binary constraints (the length of tasks excludes other assignments during their time).

- (c) (4 pt) In the table below, enforce all unary constraints by crossing out values in the table on the left below. If you made a mistake, cross out the whole table and use the right one.

F	A8	A9	A10	A11	W8	W9	W10	W11
H	A8	A9	A10	A11	W8	W9	W10	W11
P	A8	A9	A10	A11	W8	W9	W10	W11
S	A8	A9	A10	A11	W8	W9	W10	W11
T	A8	A9	A10	A11	W8	W9	W10	W11

- (d) (3 pt) Start from the table above, select the variable S and assign the value A9 to it. Perform forward checking by crossing out values in the table below. Again the table on the right is for you to use in case you believe you made a mistake.

F	A8	A9	A10	A11	W8	W9	W10	W11
H	A8	A9	A10	A11	W8	W9	W10	W11
P	A8	A9	A10	A11	W8	W9	W10	W11
S	A8	A9	A10	A11	W8	W9	W10	W11
T	A8	A9	A10	A11	W8	W9	W10	W11

Forward checking prunes the variables' domains of values inconsistent with $S = A9$, including: other choices for S, conflicting time slots for A9 (a), the choices of F that do not precede 9 (c), W from working during 9 (f), and A teaching the preschoolers (g).

- (e) (3 pt) Based on the result of (d), what variable will we choose to assign next based on the MRV heuristic (breaking ties alphabetically)? Assign the first possible value to this variable, and perform forward checking by crossing out values in the table below. Again the table on the right is for you to use in case you believe you made a mistake.

Variable F is selected and gets assigned value A8.

F	A8	A9	A10	A11	W8	W9	W10	W11
H	A8	A9	A10	A11	W8	W9	W10	W11
P	A8	A9	A10	A11	W8	W9	W10	W11
S	A8	A9	A10	A11	W8	W9	W10	W11
T	A8	A9	A10	A11	W8	W9	W10	W11

Have we arrived at a dead end (i.e., has any of the domains become empty)?

No.

- (f) (4 pt) We return to the result from enforcing just the unary constraints, which we did in (c). Select the variable S and assign the value A9. Enforce arc consistency by crossing out values in the table below.

F	A8	A9	A10	A11	W8	W9	W10	W11
H	A8	A9	A10	A11	W8	W9	W10	W11
P	A8	A9	A10	A11	W8	W9	W10	W11
S	A8	A9	A10	A11	W8	W9	W10	W11
T	A8	A9	A10	A11	W8	W9	W10	W11

- (g) (2 pt) Compare your answers to (d) and to (f). Does arc consistency remove more values or less values than forward checking does? Explain why.

Arc consistency removes more values by checking more relationships between variables: AC checks consistency between every pair of variables, and re-checks after domain pruning, while FC only checks between assigned and unassigned variables.

- (h) (1 pt) Check your answer to (f). Without backtracking, does any solution exist along this path? Provide the solution(s) or state that there is none.

AC along this path gives 1 solution: F: A8 H: A10 P: W8 S: A9 T: W10

Backtracking is unnecessary since the constraints have been enforced by arc consistency and only single values remained in each domain.

Q6. [7 pts] CSPs: Properties

- (a) [1 pt] When enforcing arc consistency in a CSP, the set of values which remain when the algorithm terminates does not depend on the order in which arcs are processed from the queue.

True False

- (b) [1 pt] In a general CSP with n variables, each taking d possible values, what is the maximum number of times a backtracking search algorithm might have to backtrack (i.e. the number of the times it generates an assignment, partial or complete, that violates the constraints) before finding a solution or concluding that none exists? (circle one)

0 $O(1)$ $O(nd^2)$ $O(n^2d^3)$ $O(d^n)$ ∞

- (c) [1 pt] What is the maximum number of times a backtracking search algorithm might have to backtrack in a general CSP, if it is running arc consistency and applying the MRV and LCV heuristics? (circle one)

0 $O(1)$ $O(nd^2)$ $O(n^2d^3)$ $O(d^n)$ ∞

- (d) [1 pt] What is the maximum number of times a backtracking search algorithm might have to backtrack in a *tree-structured* CSP, if it is running arc consistency and using an optimal variable ordering? (circle one)

0 $O(1)$ $O(nd^2)$ $O(n^2d^3)$ $O(d^n)$ ∞

Q6. [7 pts] CSPs: Properties

- (a) [1 pt] When enforcing arc consistency in a CSP, the set of values which remain when the algorithm terminates does not depend on the order in which arcs are processed from the queue.

☒ True ☐ False

- (b) [1 pt] In a general CSP with n variables, each taking d possible values, what is the maximum number of times a backtracking search algorithm might have to backtrack (i.e. the number of the times it generates an assignment, partial or complete, that violates the constraints) before finding a solution or concluding that none exists? (circle one)

0 $O(1)$ $O(nd^2)$ $O(n^2d^3)$ ☒ $O(d^n)$ ∞

In general, the search might have to examine all possible assignments.

- (c) [1 pt] What is the maximum number of times a backtracking search algorithm might have to backtrack in a general CSP, if it is running arc consistency and applying the MRV and LCV heuristics? (circle one)

0 $O(1)$ $O(nd^2)$ $O(n^2d^3)$ ☒ $O(d^n)$ ∞

The MRV and LCV heuristics are often helpful to guide the search, but are not guaranteed to reduce backtracking in the worst case.

In fact, CSP solving is NP-complete, so any polynomial-time method for solving general CSPs would constitute a proof of $P = NP$ (worth a million dollars from the Clay Mathematics Institute!).

- (d) [1 pt] What is the maximum number of times a backtracking search algorithm might have to backtrack in a *tree-structured* CSP, if it is running arc consistency and using an optimal variable ordering? (circle one)

☒ 0 $O(1)$ $O(nd^2)$ $O(n^2d^3)$ $O(d^n)$ ∞

Applying arc consistency to a tree-structured CSP guarantees that no backtracking is required, if variables are assigned starting at the root and moving down towards the leaves.

Q3. [27 pts] CSPs

(a) Pacman's new house

After years of struggling through mazes, Pacman has finally made peace with the ghosts, Blinky, Pinky, Inky, and Clyde, and invited them to live with him and Ms. Pacman. The move has forced Pacman to change the rooming assignments in his house, which has 6 rooms. He has decided to figure out the new assignments with a CSP in which the variables are Pacman (**P**), Ms. Pacman (**M**), Blinky (**B**), Pinky (**K**), Inky (**I**), and Clyde (**C**), the values are which room they will stay in, from 1-6, and the constraints are:

- i) No two agents can stay in the same room
- ii) $P > 3$
- iii) **K** is less than **P**
- iv) **M** is either 5 or 6
- v) $P > M$
- vi) **B** is even
- vii) **I** is not 1 or 6
- viii) $|I - C| = 1$
- ix) $|P - B| = 2$

- (i) [1 pt] **Unary constraints** On the grid below cross out the values from each domain that are eliminated by enforcing unary constraints.

P	1	2	3	4	5	6
B	1	2	3	4	5	6
C	1	2	3	4	5	6
K	1	2	3	4	5	6
I	1	2	3	4	5	6
M	1	2	3	4	5	6

The unary constraints are ii, iv, vi, and vii. ii crosses out 1,2, and 3 for P. iv crosses out 1,2,3,4 for M. vi crosses out 1,3, and 5 for B. vii crosses out 1 and 6 for I. K and C have no unary constraints, so their domains remain the same.

- (ii) [1 pt] **MRV** According to the Minimum Remaining Value (MRV) heuristic, which variable should be assigned to first?

☐ P ☐ B ☐ C ☐ K ☐ I ☒ M

M has the fewest value remaining in its domain (2), so it should be selected first for assignment.

- (iii) [2 pts] **Forward Checking** For the purposes of decoupling this problem from your solution to the previous problem, assume we choose to assign P first, and assign it the value 6. What are the resulting domains after enforcing unary constraints (from part i) and running forward checking for this assignment?

P						6
B	1	2	3	4	5	6
C	1	2	3	4	5	6
K	1	2	3	4	5	6
I	1	2	3	4	5	6
M	1	2	3	4	5	6

In addition to enforcing the unary constraints from part i, the domains are further constrained by all constraints involving P. This includes constraints i, iii, v, and ix. i removes 6 from the domains of all variables. iii removes 6 from the domain of K (already removed by constraint i). v removes 6 from the domain of M (also already removed by i). ix removes 2 and 6 from the domain of B.

- (iv) [3 pts] **Iterative Improvement** Instead of running backtracking search, you decide to start over and run iterative improvement with the min-conflicts heuristic for value selection. Starting with the following assignment:

P:6, B:4, C:3, K:2, I:1, M:5

First, for each variable write down how many constraints it violates in the table below.

Then, in the table on the right, for all variables that could be selected for assignment, put an x in any box

that corresponds to a possible value that could be assigned to that variable according to min-conflicts. When marking next values a variable could take on, only mark values different from the current one.

Variable	# violated		1	2	3	4	5	6
P	0	P						
B	0	B						
C	1	C		x				
K	0	K						
I	2	I		x		x		
M	0	M						

Both I and C violate constraint viii, because $|I-C|=2$. I also violates constraint vii. No other variables violate any constraints. According to iterative improvement, any conflicted variable could be selected for assignment, in this case I and C. According to min-conflicts, the values that those variables can take on are the values that minimize the number of constraints violated by the variable. Assigning 2 or 4 to I causes it to violate constraint i, because other variables already have the values 2 and 4. Assigning 2 to C also only causes C to violate 1 constraint.